



Key Files Guide

Every important file — what it does, why it exists, and when to look at it



Backend — Spring Boot Files

ENTRY POINT `src/main/java/com/wipro/csd/CsdApplication.java`

WHAT IT DOES

This is the main class of the entire Spring Boot application. It has the `public static void main()` method that starts the server. Without this file, nothing runs.

WHY IT MATTERS

If someone asks "where does the application start?" — point to this file. It's the first thing Java executes.

CONFIG `src/main/resources/application.properties`

WHAT IT DOES

All application settings live here: the server port (8080), database connection (H2), JWT secret key, and token expiry time. Spring Boot reads this file automatically on startup.

WHY IT MATTERS

This is the single place to change the database, port, or security settings. To switch to MySQL, you would only change 3 lines in this file.

SECURITY `src/main/java/com/wipro/csd/config/SecurityConfig.java`

WHAT IT DOES

Configures Spring Security for the entire application. Defines which endpoints are public (login, register) and which require authentication. Sets up JWT filter that validates the token on every request. Also configures password encryption using BCrypt.

WHY IT MATTERS

This is the backbone of all security. If you're asked "how is the API secured?" — this is the answer. Every API call (except login/register) must pass through the JWT validation filter defined here.

SEED DATA `src/main/java/com/wipro/csd/config/DataInitializer.java`

WHAT IT DOES

Runs automatically on startup (via `@PostConstruct`). Creates all demo users (1 Admin, 2 Managers, 4 Reps, 4 Customers), 10 sample tickets, 2 outages, and 4 notifications. Only runs once — skips if data

already exists.

WHY IT MATTERS

This is why the app has data immediately after running — no manual database setup needed. All passwords are encrypted using BCryptPasswordEncoder before saving.

AUTH API `src/main/java/com/wipro/csd/controller/AuthController.java`

WHAT IT DOES

Handles `POST /api/auth/login` and `POST /api/auth/register`. On successful login, generates a JWT token and returns the user object plus token to the frontend. On registration, creates a new CUSTOMER user.

WHY IT MATTERS

The entry point for all authentication. When a user clicks "Sign In", this is the Java code that runs on the server to validate credentials and issue the token.

MODEL `src/main/java/com/wipro/csd/model/User.java`

WHAT IT DOES

The JPA entity representing a user in the database. Defines all fields: id, name, email, password, role (ADMIN/MANAGER/REPRESENTATIVE/CUSTOMER), phone, location, managerId, firstLogin, createdAt. The `@Entity` annotation tells Spring to create a database table for this class.

WHY IT MATTERS

Every user in the system (admin, manager, rep, customer) is stored as a row in the Users table defined by this class.

MODEL `src/main/java/com/wipro/csd/model/Ticket.java`

WHAT IT DOES

JPA entity for support tickets. Fields include: customerId, customerName, domain, subject, description, status (OPEN/IN_PROGRESS/CLOSED), repId, repName, responseTimeHrs, resolutionTimeHrs, rating, createdAt, resolvedAt.

WHY IT MATTERS

The core business entity of the whole application. Every ticket raised by a customer, assigned to a rep, and closed gets stored here.

ANALYTICS `src/main/java/com/wipro/csd/controller/AnalyticsController.java`

WHAT IT DOES

Provides `GET /api/analytics/admin`, `/manager`, `/rep`, `/customer`. Each endpoint aggregates data (counts, averages) and returns a map of statistics that the frontend uses for the dashboard cards and charts.

WHY IT MATTERS

All the numbers on the dashboard cards (Total Customers: 4, Open Tickets: 3, Avg Satisfaction: 4.3) come from this controller.

A Frontend — Angular Key Files

ROUTING frontend/src/app/app.routes.ts

WHAT IT DOES

Defines all the URL routes of the application: `/login`, `/admin`, `/manager`, `/representative`, `/customer`, `/profile`. Each route is lazy-loaded and protected by the auth guard with a specific role requirement.

WHY IT MATTERS

This is the map of the application. If you want to add a new page, this is where you register it. The `data: { role: 'ADMIN' }` property on each route is what the auth guard reads to enforce access control.

APP CONFIG frontend/src/app/app.config.ts

WHAT IT DOES

Bootstraps the Angular application. Registers the router (with hash-based URLs), the HTTP client, and the JWT interceptor as a provider. This is the Angular equivalent of Spring's application context setup.

WHY IT MATTERS

The JWT interceptor is registered here globally — `withInterceptors([jwtInterceptor])`. Without this, no API calls would include the auth token.

JWT INTERCEPTOR frontend/src/app/interceptors/jwt.interceptor.ts

WHAT IT DOES

Intercepts every HTTP request going out of Angular. Reads the JWT token from sessionStorage and automatically adds `Authorization: Bearer <token>` to the request headers. If no token exists, the request is sent as-is (for login/register endpoints).

WHY IT MATTERS

This is what makes protected API calls work. Without the interceptor, every call to `/api/tickets`, `/api/users`, etc. would fail with 401 Unauthorized because no token is sent.

AUTH GUARD frontend/src/app/guards/auth.guard.ts

WHAT IT DOES

A function that runs before Angular renders any protected route. Checks: (1) Is there a token in sessionStorage? (2) Does the user's role match the required role for the route? If either check fails, redirects to `/login`.

WHY IT MATTERS

Prevents unauthorized users from accessing pages. This is the frontend's first line of defense — even before the backend API is called.

PROXY `frontend/proxy.conf.json`

WHAT IT DOES

Tells the Angular development server to forward all requests that start with `/api` to `http://localhost:8080`. So when Angular calls `/api/auth/login`, it silently becomes `http://localhost:8080/api/auth/login`.

WHY IT MATTERS

This solves the CORS problem. Without this, the browser would block requests from port 4200 to port 8080 as a security measure.

GLOBAL STYLES `frontend/src/styles.css`

WHAT IT DOES

The single global stylesheet for the entire application (~480 lines). Defines CSS variables (lilac theme colours), card styles, table styles, sidebar layout, button styles, modal styles, badge colours, animations, and the login page design.

WHY IT MATTERS

All visual styling lives here. If the interviewer asks about the lilac theme, animations, or layout — this is the file to point to. Key CSS variable: `--primary: #8b5cf6`.

AUTH SERVICE `frontend/src/app/services/auth.service.ts`

WHAT IT DOES

Handles all authentication logic on the frontend: `login()`, `register()`, `logout()`, `setSession()` (saves token to sessionStorage), `getUser()`, `getToken()`, `updateProfile()`, `changePassword()`.

WHY IT MATTERS

The single source of truth for auth state on the frontend. Every component that needs to know who is logged in uses this service.



Page Components

PAGE `frontend/src/app/pages/login/login.component.ts + .html`

WHAT IT DOES

The full-page lilac login screen with Sign In / Sign Up cards. Clicking a card triggers a smooth CSS animation filling the page with the appropriate form. Sign In calls `AuthService.login()` and redirects by role. Sign Up calls `AuthService.register()`.

PAGE `frontend/src/app/pages/admin/admin.component.ts + .html`

WHAT IT DOES

Admin dashboard with four tabs (Dashboard, Employees, Tickets, Outages) controlled by Angular query params. Uses Chart.js for bar and doughnut charts. Includes modals for adding/editing employees and reporting outages.

PAGE `frontend/src/app/pages/representative/representative.component.ts + .html`

WHAT IT DOES

Rep dashboard with three tabs (Dashboard, My Tickets, Search). The ticket update modal changes ticket status from OPEN → IN_PROGRESS → CLOSED. The search feature uses a debounced input (400ms delay) to search customers by name, phone, or ID.

PAGE `frontend/src/app/pages/customer/customer.component.ts + .html`

WHAT IT DOES

Customer dashboard with two tabs (Dashboard, My Tickets). Dashboard shows stats, outage alert (if any in their city), plans, and About section. My Tickets tab shows ticket history with the ability to rate closed tickets and raise new ones.

SHARED `frontend/src/app/shared/sidebar/sidebar.component.ts + .html`

WHAT IT DOES

The left navigation sidebar shared by all dashboards. Reads the logged-in user's role to display role-specific navigation items. All navigation uses Angular Router with query params. The avatar at the bottom

logs the user out on click. Live Support routes through ChatService to open the chatbot.

SHARED `frontend/src/app/shared/chatbot/chatbot.component.ts + .html`

WHAT IT DOES

The floating  chat assistant. Opens via a ChatService observable (so the sidebar can trigger it). Has 12 keyword-response pairs covering common customer issues. Unknown messages fall back to a ticket-raising suggestion. Auto-scrolls to the latest message.